Week 11: Heuristic Optimization & Constraint Propagation in CSP

Objective

To implement advanced CSP solving strategies using **heuristic-driven search** and **constraint propagation techniques**, and apply them to complex combinatorial problems.

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs after running.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the .ipynb** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the obtained marks.**

Task 1: MRV-Based Variable Selection Strategy

Implement a CSP solver for the **4-Queens Problem** using the **Minimum Remaining Values (MRV)** heuristic.

Requirements:

- At each step:
  - Select variable with smallest domain size
- Domains must be dynamically updated during search
- Do NOT use forward checking or AC-3
- Variables: $X1, X2, X3, X4$(columns of 4-Queens)
- Domains:

  D(Xi) = {1,2,3,4}
- Constraints:
  - No same row
  - No diagonal conflict

Expected Output:

- Solution to 4-Queens
- Log showing variable selection order

Task 2: Implementation of Least Constraining Value (LCV) Heuristic for Map Coloring CSP

Problem Definition

You are given the following CSP:

- **Variables**:
$$X = \{WA, NT, SA, Q, NSW, V\}$$
- **Domains**:
$$D(X_i) = \{Red, Green, Blue\}$$
- **Constraints (Binary Inequality Constraints)**:
  - WA ≠ NT
  - WA ≠ SA
  - NT ≠ SA
  - NT ≠ Q
  - SA ≠ Q
  - SA ≠ NSW
  - SA ≠ V
  - Q ≠ NSW
  - NSW ≠ V

Implementation Requirements

You are required to:

1. Implement a CSP solver using:
   - Backtracking search
2. Integrate **Least Constraining Value (LCV)** heuristic:
   - For a selected variable:
     - Evaluate each value in its domain
     - Count how many values it eliminates from neighboring variables
     - Sort values in ascending order of elimination
3. Do NOT use:

- MRV
- Forward Checking
- AC-3

Expected Output

- For each variable assignment, display:

Variable Selected: SA
LCV Order: [Green, Blue, Red]

- Final valid assignment:

WA=Red, NT=Green, SA=Blue, Q=Red, NSW=Green, V=Red

Task 3: Implementation of Arc Consistency (AC-3) Algorithm on a Binary CSP

Problem Definition

You are given the following CSP:

- **Variables**:

$$X = \{X1, X2, X3\}$$

- **Domains**:

D(X1) = {1, 2, 3, 4}
D(X2) = {1, 2, 3, 4}
D(X3) = {1, 2, 3, 4}

- **Constraints**:
  - X1 < X2
  - X2 < X3

Implementation Requirements

You are required to:

1. Implement the **AC-3 algorithm**:
   - Initialize a queue with all arcs:

(X1, X2), (X2, X1), (X2, X3), (X3, X2)

2. Implement **Revise(Xi, Xj)**:
   - Remove values from $D(Xi)$that have no supporting value in $D(Xj)$

3. Continue until:
    - Queue becomes empty OR
    - Any domain becomes empty

Expected Output

- Step-by-step revision log:

Revise(X1, X2): Removed 4
Revise(X2, X3): Removed 1

- Final domains:

D(X1) = {1, 2}
D(X2) = {2, 3}
D(X3) = {3, 4}

Task 4: Arc Consistency on a Partially Filled Sudoku Grid

Problem Definition

You are given the following **4×4 Sudoku grid** (simplified for implementation):

[1, 0, 0, 4]
[0, 0, 3, 0]
[0, 3, 0, 0]
[2, 0, 0, 1]

Requirements

- Variables:
    - Each empty cell
- Domains:
    - {1, 2, 3, 4}
- Constraints:
    - All-different for:
        - Rows
        - Columns
        - 2×2 subgrids

Implementation Requirements:

- Apply **AC-3 algorithm only**
- Do NOT perform backtracking
- Reduce domains of all unassigned variables

Expected Output

- Domain representation for each empty cell:

Cell (0,1) → {2,3}
Cell (1,0) → {4}
...

Task 5: Cryptarithmetic CSP Modeling

Formally model the problem:

SEND + MORE = MONEY

Requirements:

- Define:
  - Variables: S, E, N, D, M, O, R, Y, carry variables
- Domains:
  - Digits {0–9}
- Constraints:
  - All-different constraint
  - Column-wise arithmetic constraints
  - Leading digit constraints

Expected Output:

- Formal CSP specification
- Constraint equations representation

Task 6: Cryptarithmetic Solver with Heuristics

Implement a complete CSP solver for:

TWO + TWO = FOUR

Requirements:

- Use:

    - Backtracking
    - MRV heuristic
    - LCV heuristic
- Enforce:
    - All-different constraint
    - Valid arithmetic constraints

Additional Constraints:

- Leading digits must not be zero
- Each variable must be assigned a unique digit

Expected Output:

T=7, W=3, O=4, F=1, U=6, R=8

Task 7: Intelligent Exam Timetabling using MRV & LCV

Scenario:

A university must generate an **exam timetable** minimizing scheduling conflicts among courses.

Requirements:

Model this as a CSP.

- Variables:
    - Each exam E_i
- Domains:
    - Time slots: {Morning, Afternoon, Evening}
- Constraints:
    - Exams with common students cannot be scheduled in the same slot
    - Limited number of rooms per time slot (capacity constraint)
    - Some exams must be scheduled earlier due to administrative constraints

Implementation Requirements:

- Implement CSP solver using:
    - Backtracking

- **MRV heuristic (variable selection)**
- **LCV heuristic (value ordering)**

Expected Output:

- Exam schedule:

E1 → Morning
E2 → Afternoon
E3 → Evening

- Logs:
  - Variable selection using MRV
  - Value ordering using LCV

Task 8: Network Frequency Assignment using Arc Consistency (AC-3)

Scenario:

A telecom company must assign **radio frequencies to communication towers** such that interference is minimized.

Requirements:

Model this as a CSP.

- Variables:
  - Each tower T_i
- Domains:
  - Frequencies: {F1, F2, F3, F4}
- Constraints:
  - Adjacent towers must not use the same frequency
  - Certain towers cannot use specific frequencies (hardware limitations)

Implementation Requirements:

- Implement:
  - **AC-3 algorithm**
- Apply arc consistency:
  - Before any search (preprocessing step)

Expected Output:

- Reduced domains for each tower:

T1 → {F2, F3}
T2 → {F1, F4}

- Revision logs:

Arc (T1, T2): Removed F1 from T1